VANIER
CÉGEP/COLLEGE

FINAL PROJECT

Building a Recommendation Engine with Association Rules

Boréal Marché - from 1,000,000 baskets to a live application, using Apriori

Item	Detail
Course	420-975-VA — Unsupervised Learning
Required algorithm	Apriori (mlxtend)
Dataset	1,000,000 baskets · 8,379,837 rows · 98 products
Deliverable	Jupyter notebook, generated rules, streamlit app (python code)
Due date	Wednesday 4 November 2026, 23:59 (EST)
Work	Individual

1 · The brief

Boréal Marché is a Québec online grocery retailer. Today its website shows the same list of “popular products” to every customer. Management wants to replace that list with real recommendations based on what is already in the shopping basket, and it wants to understand why each suggestion is made.

You are given one full year of transactions. You must deliver a complete system: from the raw file all the way to a web application you could demonstrate in front of a client.

> **The question your system answers:** “Given what this customer has already put in their basket, what should we suggest next - and why?”

2 · The deliverable

You submit a folder containing the structure below.

```
boreal-reco-<yourname>/
├── README.md                  ← overview, results, limitations, screenshots
├── requirements.txt
├── data/
│   └── transactions.zip       (or download instructions)
├── notebooks/
│   └── 01_mining.ipynb        ← EXECUTED, with outputs visible
├── models/
│   ├── rules.json             ← rules mined offline
│   └── metadata.json          ← thresholds, mining date, metrics
├── app/
│   └── streamlit_app.py       ← the application
```

```
└── reports/
    └── benchmark.md          ← your timing and memory measurements
```

> **The notebook must be submitted EXECUTED, with every output visible**.

3 · The data

File	Contents	Size
`transactions.zip`	TransactionID, Date, CustomerID, Item	36 MB zipped · 304 MB unzipped
`products.csv`	Item, Item_FR, Category, Price (98 products)	4 KB

The transaction file is in long format: one row per item, not one row per basket. Pandas reads the ZIP archive directly, there is no need to unzip it by hand:

```
import pandas as pd
tx = pd.read_csv('data/transactions.zip', usecols=['TransactionID','Item'])
# 8,379,837 rows · 1,000,000 baskets · 98 products · 8.4 items per basket
```

> **Your first real difficulty.** Apriori needs a boolean baskets × products table. Going from long format to wide format is your first genuine engineering problem: a naive approach can exhaust memory or take several minutes. Measure it and document the approach you chose.

4 · The pipeline to build

Six phases. Each one must be visible in the notebook or in the repository.

Phase 1 — Load and prepare (the wide format)

- Load the transactions and verify number of baskets, number of products, average basket size.
- Build the boolean table (1,000,000 × 98). Measure the time taken and the memory it occupies.
- Compute how often each product appears. Show the 10 most frequent and the 5 rarest.
- **Before mining anything, identify at least one product that is going to cause trouble, and write your prediction in the notebook.**
- Find two products in the catalogue that are really the same product recorded twice. Decide whether to merge them and justify your decision.

Phase 2 — Mine the rules with Apriori

You must use `mlxtend.frequent_patterns.apriori`. FP-Growth, ECLAT and any other implementation are not permitted: the point of this project is precisely to meet the limits of Apriori and document them.

```
from mlxtend.frequent_patterns import apriori, association_rules

itemsets = apriori(basket, min_support=?, use_colnames=True, low_memory=?)
rules    = association_rules(itemsets, metric='confidence', min_threshold=?)
```

- Test at least four values of min_support and fill in the table in Phase 6.
- For each run, record: number of itemsets, number of rules, time taken, peak memory.
- **Warning: on 1,000,000 baskets, Apriori with its default settings fails.** You will hit a `MemoryError`. This is intentional. Your job is to understand why, find the parameter that fixes it, and explain both in `benchmark.md`.

> **Survival tip.** Start at `min_support = 0.05` and work downwards. Watch your memory. Save your notebook before each more aggressive run: a killed process loses everything you have not written to disk.

Phase 3 — Evaluate the rules

- Define support, confidence and lift in one plain sentence each — no formulas.
- Sort the simple rules (one item on each side) by **confidence** and show the top 10.
- Sort the same rules by **lift** and show the top 10.
- **Compare the two lists and explain the difference.** Was your Phase 1 prediction correct? This is the most heavily weighted question in the project.
- Keep only the rules with lift > 1. Justify that filter.

Phase 4 — Seasonal analysis (required)

The data covers a full year. Some behaviour is seasonal.

- Mine the rules separately on June–August, then on December–February.
- Compare the support, confidence and lift of the same rules across both seasons.
- Show at least one rule that exists in one season and disappears entirely in the other.
- **Conclude: how often should Boréal Marché re-mine its rules, and why? What happens if the website serves year-round rules in January?**

Phase 5 — Save the rules (separate offline from online)

- Save the filtered rules to `models/rules.json`.
- Save to `models/metadata.json`: the thresholds you chose, the mining date, the number of rules, the coverage.
- **The application (streamlit) must NEVER call Apriori.** It loads a file. Mining is expensive and offline; recommending is instant and online.

Phase 6 — Measure and document

Fill in this table in reports/benchmark.md with your own measurements:

min_support	Itemsets	Rules	Time	Peak memory	Outcome
0.05					
0.02					
0.01					
0.005					

- Compute coverage: how many of the 98 products can your system ever recommend?
- Name two things this approach cannot do, and what you would add to fix each one.

VANIER
CÉGEP / COLLEGE

5 · The Streamlit application

The application simulates the experience of a customer on the website. It must run with:

```
streamlit run app/streamlit_app.py
```

The screenshots that follow show the expected behaviour. You do not have to copy this visual style — but every piece of information visible here must appear in your application.

5.1 — Three required zones

Zone	Required contents
1 · Catalogue	All 98 products · filter by category · search by name · “Add” button
2 · Basket	Chosen items with price and category · remove an item · clear the basket · total
3 · Recommendations	Up to 5 suggestions, sorted by descending lift, each with details and an explanation

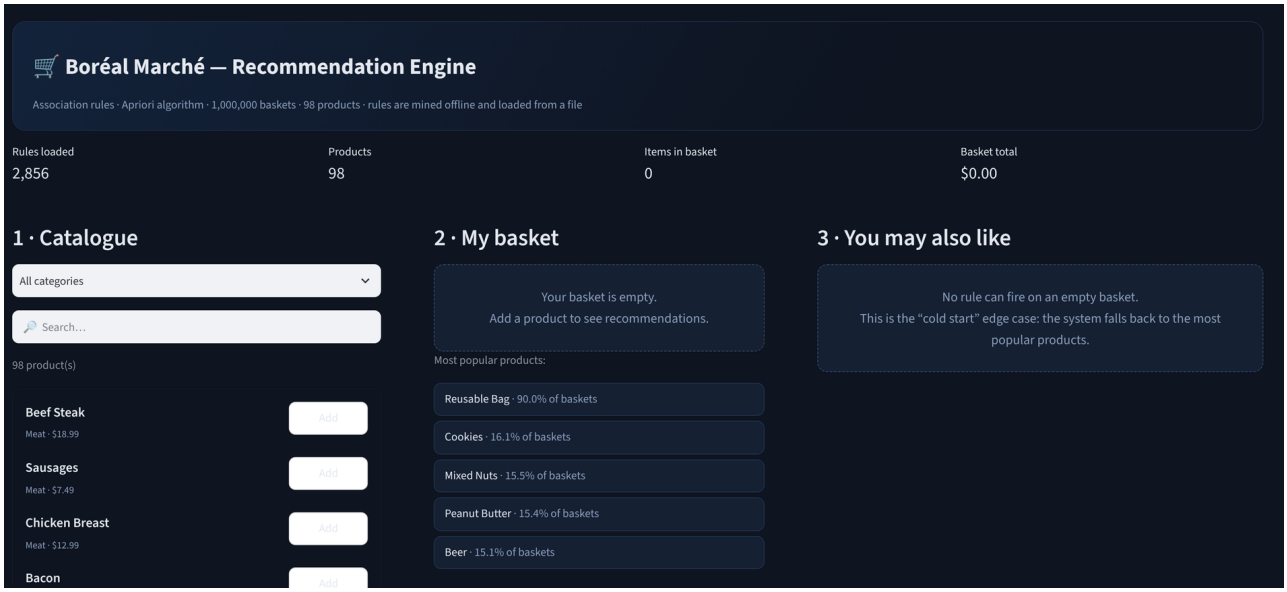


Figure 1 — Empty basket. No rule can fire, so the system falls back to the most popular products (the “cold start” edge case).

5.2 — Every recommendation must display

- **Product details:** name, category, price.
- **A plain-language comment:** “Because your basket contains X, customers who buy that also buy Y N% of the time — that is L× more often than average.”
- **The three metrics:** support, confidence, lift.
- **A strength label** derived from the lift (weak / moderate / strong / very strong).

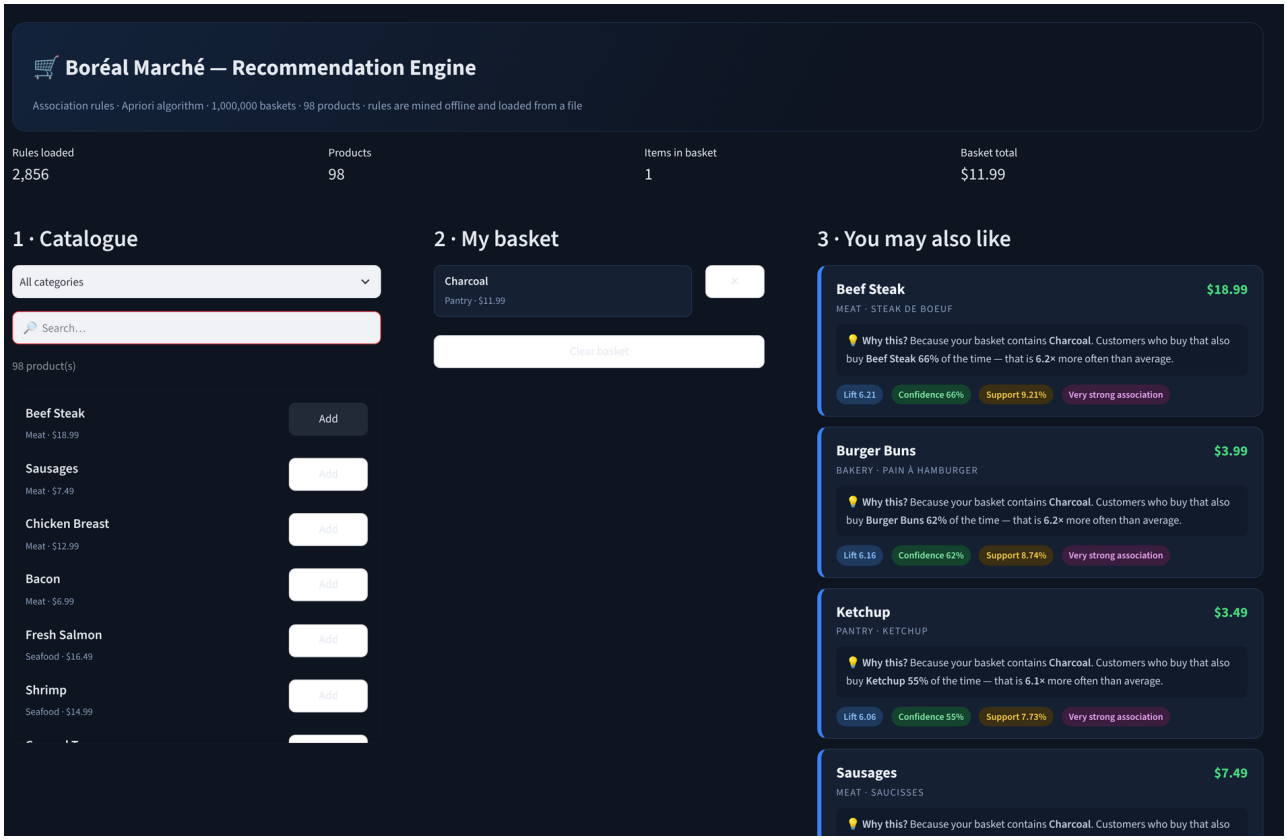


Figure 2 — One item in the basket (Charcoal). Suggestions are sorted by lift, each with its price, its "Why this?" explanation and its three metrics.

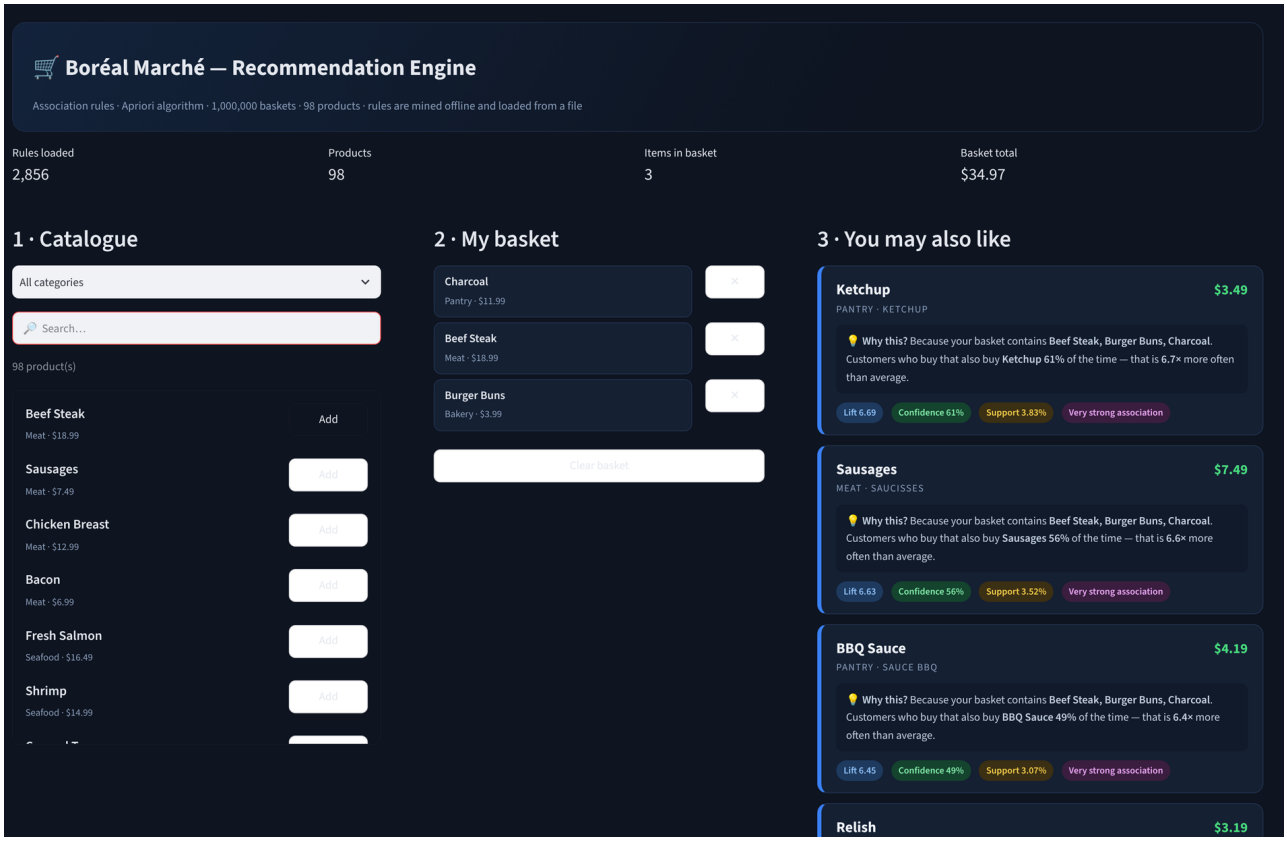


Figure 3 — A three-item basket. Rules with multi-item antecedents now fire: the explanation names every triggering item, and items already in the basket are never suggested again.

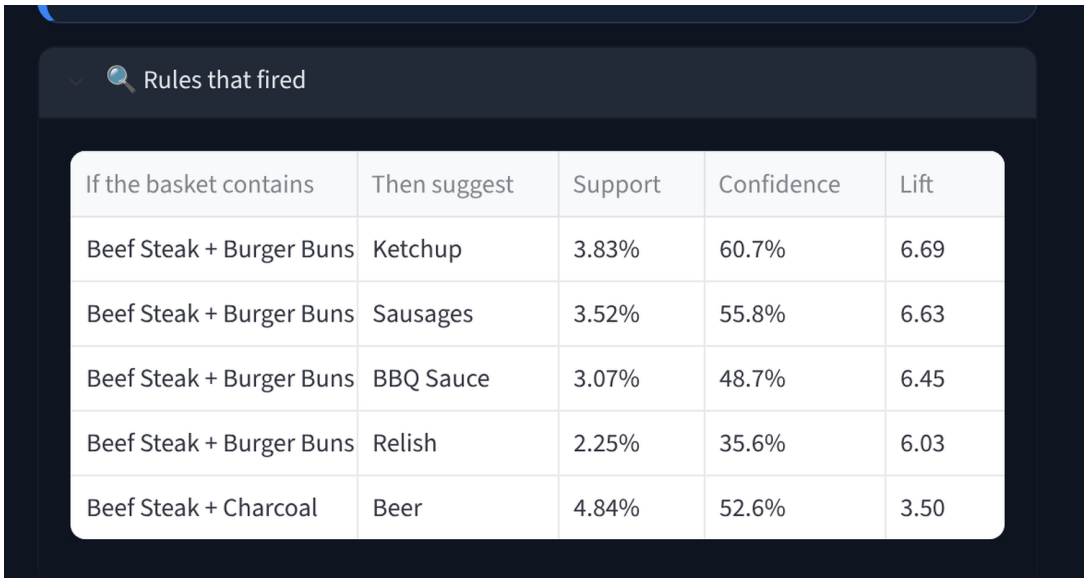


Figure 4 — The expandable "Rules that fired" section: the table of rules behind the suggestions shown. This is required — it is what makes the system auditable.

5.3 — Edge cases to handle (graded)

Situation	Expected behaviour
Empty basket	A clear message plus a fallback to popular products
No rule fires	An explicit message — not a blank panel — then a fallback
Item already in the basket	Never suggested again
Two rules suggest the same item	Keep the one with the higher lift
A very rare product	Explain why it appears in no rule at all

VANIER
CÉGEP/COLLEGE

6 · Grading

#	Criterion	Points
1	Data preparation: loading, wide format, frequencies, prediction written in advance	10
2	Apriori: min_support runs, time and memory measured, MemoryError understood and solved	20
3	Rule evaluation: support / confidence / lift, and the comparison of both rankings	20
4	Seasonal analysis and the conclusion on re-mining frequency	10
5	Recommendation engine: subset matching, lift ranking, edge cases	15
6	Streamlit application: three zones, details, explanations, metrics	15
7	Repository and README: structure, reproducibility, honesty about limitations	10
	TOTAL	100

> **Honesty bonus (up to 5 points).** A student who falls into a trap, notices it, and documents the correction scores higher than one who presents a clean result without showing their reasoning. Documented negative results count.

7 · Schedule — 12 days

Period	Milestone	Check
Fri 23 – Mon 26 Oct	Phase 1: data loaded, wide format built, frequencies computed	The boolean table (1,000,000 × 98) exists
Tue 27 – Thu 29 Oct	Phase 2: Apriori, threshold runs, benchmark filled in	benchmark.md holds real measurements
Fri 30 Oct – Sun 1 Nov	Phases 3 and 4: rule evaluation and seasonal analysis	`rules.json is saved`
Mon 2 – Tue 3 Nov	Phases 5 and 6: Streamlit application and edge cases	The app starts and recommends
Tue 3 Nov (evening)	README, screenshots, final repository review	The repository is complete
Wed 4 Nov, 23:59	SUBMISSION: a single Git repository URL	—

> **Studio session, Tuesday 27 October:** bring your work-in-progress `benchmark.md`. If you have not managed to run Apriori even once by then, you are behind — come and say so rather than waiting.
>
> **Late submissions:** −10% per day started; nothing accepted after 7 November.

8 · Ethical and legal framing

A recommendation engine is an automated decision-making system. Three requirements:

- **Law 25 (Québec).** Any decision based exclusively on automated processing must be explainable to the person concerned. That is the whole reason for the “Rules that fired” section: your system must be able to justify every suggestion.
- **Correlation is not causation.** One of your rules will reproduce a famous example from retail analytics. Identify it in the README and explain why it proves no causal link whatsoever.
- **Reinforcement bias.** Recommending only what is already popular locks down the catalogue. Your coverage measurement exists to make that risk visible.

9 · Checklist before you submit

- The repository is public and the URL works in a private browsing window.
- The notebook is executed and every output is visible.
- benchmark.md contains real measurements, not empty cells.
- rules.json and metadata.json exist and are loaded by the application.
- The application starts with a single command and mines nothing.
- All five edge cases are tested and the README shows it.
- The README names at least one real limitation of the system.
- requirements.txt makes the environment reproducible.

> **An honest system with documented limitations is worth more than a flawless demo that hides what does not work.** That is also what an employer will ask you in an interview: not “does it work?”, but “where does it break, and how do you know?”